Mathematical Formulation of the Porous Navier-Stokes Solvers

Antigravity AI Project Authors

April 13, 2026

1 Introduction

This document outlines the formulation of the Orthogonal Sub-Grid Scales (OSGS) stabilization method and the nonlinear globalization strategy for the generalized porous Navier–Stokes equations. The algorithmic structure strictly separates the continuous finite element operator assembly from the discrete algebraic safeguarding mechanisms.

2 Solver Parameters

All bounds mapping to the internal runtime configuration are classified below.

Table 1: Nonlinear and Staggered Solver Configuration Parameters

Symbol	**Code Property**	**Description**
N_{OSGS}	`stab_cfg.osgs_iterations`	Maximum number of outer staggered loops permitted.
N_{inner}	`stab_cfg.osgs_inner_newton_iters`	Iteration limit for each inner monolithic Newton-Raphson solve.
N_{inc}	`sol_cfg.max_increases`	Allowable successive iterations where the scalar merit function expands.
N_{plateau}	`mms_cfg.require_consecutive_passes`	Number of consecutive error verifications achieving steady reduction.
$\alpha_{\min}$	`sol_cfg.linesearch_alpha_min`	Minimum allowable geometric step size.
c_1	`sol_cfg.armijo_c1`	Armijo condition parameter evaluating sufficient residual descent.
F_{div}	`sol_cfg.divergence_merit_factor`	Expansion threshold evaluating catastrophic divergence bounds.
τ_{OSGS}	`stab_cfg.osgs_tolerance`	Tolerance evaluating acceptable outer state continuity.
τ_{proj}	`stab_cfg.osgs_projection_tol`	Tolerance evaluating sub-grid spatial projection limits.
τ_{ftol}	`sol_cfg.ftol`	Terminal nonlinear residual tolerance bounding the sequence ℓ^∞ norm.
τ_{xtol}	`sol_cfg.xtol`	Coordinate step norm limit halting sequence updates.
$\tau_{\text{stagnation}}$	`sol_cfg.stagnation_noise_floor`	Noise-floor limit preventing spurious machine-epsilon stalls.
τ_{error}	`mms_cfg.tau_err`	Plateau relative error bound verifying MMS limits.

3 Algorithmic Structure

Let the primary velocity-pressure state be $U_h = (\boldsymbol{u}_h, p_h) \in \mathcal{X}_h$. The continuous sub-grid projection variables are $\boldsymbol{\pi}_h = (\boldsymbol{\pi}_{h,u}, \pi_{h,p})^\top \in \mathcal{X}_h$.

To natively balance the numerical disparity between the momentum and mass conservation equations without *a priori* macroscopic scaling, we extract an element-wise scaling vector from the exact Jacobian at each iteration: $w_k = \max(|[\mathcal{J}]_{kk}|, \epsilon_{\text{mach}}\|\text{diag}(\mathcal{J})\|_\infty, \epsilon_{\text{mach}})$. The diagonally equilibrated dimensionless merit function is defined as:

$$\Phi_{\boldsymbol{w}}(\boldsymbol{x}) = \frac{1}{2}\sum_{k=1}^{N}\left(\frac{b_k(\boldsymbol{x})}{w_k}\right)^2 \tag{1}$$

Algorithm 1: Algorithm A: Safeguarded Exact Newton-Raphson Subsolver

```
Input: Initial guess x, Operator yielding (b(x), J(x)), dynamic tolerance τ.
Initialize x_best = x, b_best = ||b(x)||_∞;
I_up = 0;
if ||b(x)||_∞ ≤ τ then
    return x (Exit: initial_ftol);
end
for i = 1, ..., N_inner do
    Compute Jacobian J_i = J(x) and residual b_i = b(x);
    Update weights w = max(|diag(J_i)|, ε_mach ||diag(J_i)||_∞, ε_mach);
    Evaluate Φ_curr = Φ_w(x);
    try {
        Solve linear system J_i δx = b_i;
    } catch {
        Restore x ← x_best. return (Exit: linear_solve_exception);
    }
    if δx contains NaNs then
        Restore x ← x_best. return (Exit: linear_solve_nan);
    end
    /* Exact analytical directional derivative (D = −b_i^⊤ w^{−2} J_i δx = −2Φ_curr)  */
    D = −2.0 · Φ_curr;
    α = 1.0, S_step_accepted = false;
    for j = 1, ..., 15 do
        x_trial = x − αδx;
        Compute b̂ = b(x_trial) and Φ̂_w = Φ_w(x_trial);
        if not isnan(Φ̂_w) and Φ̂_w ≤ Φ_curr + c_1 α D then
            S_step_accepted = true. break;
        end
        α = 0.5α;
        if α ≤ α_min then
            break;
        end
    end
    if not S_step_accepted then
        Restore x ← x_best. return (Exit: linesearch_failed);
    end
    x ← x_trial ;                                                  // Accept step
    if ||b̂||_∞ < b_best then
        x_best = x, b_best = ||b̂||_∞;
    end
    /* Terminal Verifications                                                  */
    if ||b̂||_∞ ≤ τ then
        return x (Exit: ftol_reached);
    end
    if α||δx||_∞ ≤ τ_xtol then
        return x (Exit: xtol_stagnation);
    end
    if Φ̂_w > Φ_curr · F_div then
        I_up = I_up + 1;
        if I_up ≥ N_inc then
            Restore x ← x_best. return (Exit: merit_divergence_escaped);
        end
    end
    else
        I_up = 0;
    end
end
return x_best;
```

Algorithm 2: Algorithm B: Canonical ASGS Baseline Solver

Input: Initial array $\boldsymbol{x}^0$, full continuous algebraic system definitions.
Configure parameters zeroing sub-grid scales: $\boldsymbol{\pi}_h = \mathbf{0}$;
Attempt Newton resolution via Algorithm A targeting τ_{ftol};
if *Algorithm A aborts or diverges* **then**
  Compute standard Picard smoothing iteration to bypass ill-conditioned regions;
  Re-evaluate Newton resolution via Algorithm A targeting τ_{ftol};
end
return Accepted converged baseline state U_h^{ASGS};

Algorithm 3: Algorithm C: OSGS Lagged Projection Refinement

Input: Accepted U_h^{ASGS} and invariant mass matrices M_u, M_p.
Initialize $U_h^0 = U_h^{\text{ASGS}}$;
Bootstrap initial sub-grid projections: $\boldsymbol{\pi}_{h,u}^0 = \Pi_{\boldsymbol{V}}(\mathcal{R}_u(U_h^0))$ and $\pi_{h,p}^0 = \Pi_Q(\mathcal{R}_p(U_h^0))$;
for $m = 1, \ldots, N_{OSGS}$ **do**
  Set inner tolerance limit: $\tau_{\text{inner}}^m = (m \leq 2)\ ?\ \max(\tau_{\text{ftol}}, 10^{-3}) : \tau_{\text{ftol}}$;
  Set $\boldsymbol{\pi}_h = \boldsymbol{\pi}_h^{m-1}$ fixed, resolve continuous nonlinear problem yielding U_h^m using Algorithm A solving to τ_{inner}^m;
  `/* Evaluate continuous spatial state drift exactly via functional integration: */`
  $d_{\text{state}}^m = \max\left(\|\boldsymbol{u}_h^m - \boldsymbol{u}_h^{m-1}\|_{L^2(\Omega)},\ \|p_h^m - p_h^{m-1}\|_{L^2(\Omega)}\right)$;
  Update continuous evaluations: $\boldsymbol{\mathcal{R}}_u^m = \mathcal{R}_u(U_h^m)$ and $\mathcal{R}_p^m = \mathcal{R}_p(U_h^m)$;
  Update continuous orthogonal projections: $(\boldsymbol{\pi}_{h,u}^m, \pi_{h,p}^m) = \left(\Pi_{\boldsymbol{V}}(\boldsymbol{\mathcal{R}}_u^m), \Pi_Q(\mathcal{R}_p^m)\right)$;
  `/* Independent Block-Weighted Anderson Acceleration */`
  (Optional) Filter limits through separated bounds using L^2 mass matrices:

$$\boldsymbol{\pi}_{h,u}^m \leftarrow \text{Anderson}(\text{History}_u, \boldsymbol{\pi}_{h,u}^{m-1}, \boldsymbol{\pi}_{h,u}^m, M_u)$$
$$\pi_{h,p}^m \leftarrow \text{Anderson}(\text{History}_p, \pi_{h,p}^{m-1}, \pi_{h,p}^m, M_p)$$

  `/* Define orthogonal projection drift explicitly evaluating algebraic block arrays: */`
  $d_\pi^m = \max\left(\sqrt{(\delta\boldsymbol{\pi}_{h,u}^m)^\top M_u \delta\boldsymbol{\pi}_{h,u}^m},\ \sqrt{(\delta\pi_{h,p}^m)^\top M_p \delta\pi_{h,p}^m}\right)$;
  if $(d_{state}^m \leq \max(\tau_{OSGS}, \tau_{stagnation}))$ **AND/OR** $(d_\pi^m \leq \max(\tau_{proj}, \tau_{stagnation}))$ **then**
    return Validated staggered boundary U_h^m;
  end
end

Algorithm 4: Algorithm D: MMS Plateau Verification Wrapper

Input: Analytical field distributions U_{exact}, accepted convergence coordinate bounds $E_{L^2}^{m-1}$.
Compute continuous tracking errors via explicit integrals: $E_{\boldsymbol{u},L^2}^m = \|\boldsymbol{u}_h^m - \boldsymbol{u}_{\text{ex}}\|_{L^2(\Omega)}$, $E_{p,L^2}^m = \|p_h^m - p_{\text{ex}}\|_{L^2(\Omega)}$;
Compute maximum relative error ratios: $r_{\max} = \max\left(\frac{|E_{\boldsymbol{u},L^2}^m - E_{\boldsymbol{u},L^2}^{m-1}|}{\max(E_{\boldsymbol{u},L^2}^m, E_{\boldsymbol{u},L^2}^{m-1}, \epsilon_{\text{err}})},\ \ldots\right)$;
if $r_{max} \leq \tau_{error}$ **then**
  $I_{\text{passes}} = I_{\text{passes}} + 1$;
end
else
  $I_{\text{passes}} = 0$;
end
if $I_{passes} \geq N_{plateau}$ **then**
  return `mms_plateau_satisfied`;
end